

Et voici la version ultra courte des **commandes vraiment utiles** :

```
# Commandes utiles
instance_public_ip=$(terraform output instance_public_ip)
instance_public_ip=$(echo $instance_public_ip | tr -d '"')
echo $instance_public_ip
ssh -i ~/.ssh/aws-ci-cd-deploy ubuntu@$instance_public_ip
cd openvpn-config-file/
scp -i ~/.ssh/aws-ci-cd-deploy
ubuntu@$instance_public_ip:/home/ubuntu/openvpn-config-file/* .
cp *.ovpn /mnt/d/wsl-files/Ubuntu/sdu/openvpn-config-file
```

```
# main.tf : contenant tout
# Bloc principal Terraform
terraform {
  # Providers nécessaires au projet
  required_providers {
    # Provider AWS officiel HashiCorp
    aws = {
      # Source du provider AWS
      source = "hashicorp/aws"

      # Version du provider AWS
      # version = "~> 5.92"
      version = "~> 6.40"
    }
  }

  # Version minimale de Terraform requise
  required_version = ">= 1.2"
}

# Variable pour la clé d'accès AWS
variable "access_key" {
  # Type de la variable
  type = string
}

# Variable pour la clé secrète AWS
variable "secret_key" {
  # Type de la variable
  type = string
}

# Variable pour l'IP à autoriser
variable "saz_ip" {
  # Type de la variable
  type = string
}
```

```

# Configuration du provider AWS
provider "aws" {
  # Région AWS utilisée
  region = var.aws_region

  # Clé d'accès AWS
  access_key = var.access_key

  # Clé secrète AWS
  secret_key = var.secret_key
}

# Variable pour l'IP à autoriser
variable "aws_region" {
  # Type de la variable
  type = string
}

# Variable pour l'IP à autoriser
variable "ec2_hostname" {
  # Type de la variable
  type = string
}

# Variable pour l'IP à autoriser
variable "app_name" {
  # Type de la variable
  type = string
}

## Définition des utilisateurs OpenVPN via une liste de maps (list of
objects)
variable "openvpn_clients" {
  type = list(object({
    openvpn_username = string
    openvpn_password = string
  }))
  sensitive = true
}

# Variable pour définir le port d'openvpn
variable "openvpn_port" {
  # Type de la variable
  type = number
}

# Recherche de l'image Ubuntu la plus récente
data "aws_ami" "ubuntu" {
  # Prendre l'AMI la plus récente correspondant au filtre
  most_recent = true
}

```

```

# Filtre sur le nom de l'AMI
filter {
  # Filtrage sur l'attribut name
  name = "name"

  # Ubuntu 24.04 Noble amd64 sur gp3
  values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-
server-*"]
}

# Limiter la recherche aux AMI officielles Canonical
owners = ["099720109477"]
}

# Récupération du VPC par défaut
data "aws_vpc" "default" {
  # Demande le VPC par défaut
  # vpc-0a6e5074c2755a2b7
  default = true
}

# Création du security group du serveur applicatif
resource "aws_security_group" "app_server_live_sg" {
  # Nom du security group
  name = var.ec2_hostname

  # Description du security group
  description = "allow inbound traffic on port SSH & openvpn 1194 and
allow all outbound"

  # Association au VPC par défaut
  vpc_id = data.aws_vpc.default.id

  # Autoriser SSH depuis n'importe quelle IP
  ingress {
    # Description de la règle
    description = "Allow ssh (port 22) from all ip adress"

    # Port source
    from_port = 22

    # Port destination
    to_port = 22

    # Protocole utilisé
    protocol = "tcp"

    # Autoriser depuis toutes les IP
    cidr_blocks = [var.saz_ip]
  }

  # Autoriser port openvpn (port TCP ${var.openvpn_port}) depuis

```

```

n'importe quelle IP
ingress {
    # Description de la règle
    description = "Allow openvpn (port TCP ${var.openvpn_port}) from any
ip adress"

    # Port source
    from_port = var.openvpn_port

    # Port destination
    to_port = var.openvpn_port

    # Protocole utilisé
    protocol = "tcp"

    # Autoriser depuis toutes les IP
    cidr_blocks = ["0.0.0.0/0"]
}

ingress {
    # Description de la règle
    description = "Allow openvpn (port UDP ${var.openvpn_port}) from any
ip adress"

    # Port source
    from_port = var.openvpn_port

    # Port destination
    to_port = var.openvpn_port

    # Protocole utilisé
    protocol = "udp"

    # Autoriser depuis toutes les IP
    cidr_blocks = ["0.0.0.0/0"]
}

# Autoriser tout le trafic sortant
egress {
    # Description de la règle
    description = "Allow all outbound traffic"

    # Tous les ports
    from_port = 0

    # Tous les ports
    to_port = 0

    # Tous les protocoles
    protocol = "-1"

    # Autoriser vers toutes les IP
    cidr_blocks = ["0.0.0.0/0"]
}

```

```

}

}

# Création de l'instance EC2
resource "aws_instance" "app_server_live" {
  # ID de l'AMI Ubuntu récupérée plus haut
  ami = data.aws_ami.ubuntu.id

  # Type d'instance EC2
  instance_type = "t3.micro"

  # Nom de la clé SSH AWS à utiliser
  key_name = "aws-ci-cd-deploy"

  # Association du security group à l'instance
  vpc_security_group_ids = [aws_security_group.app_server_live_sg.id]

  ## Encoder le script en pour AWSOpenvpn_clients
  user_data = templatefile("${path.module}/templates/install-openvpn-
server.sh", {
    openvpn_clients = var.openvpn_clients
    ec2_hostname = var.ec2_hostname
    openvpn_port = var.openvpn_port
  })

  # Tags de l'instance
  tags = {
    # Nom affiché dans AWS pour l'instance
    Name = var.ec2_hostname
  }
}

# Afficher l'IP publique de l'instance créée
output "instance_public_ip" {
  # Description de la valeur affichée
  description = "Adresse IP publique de l'instance"

  # IP publique de l'instance EC2
  value = aws_instance.app_server_live.public_ip
}

```

PROF

Using Gitlab CICD variables in Terraform

<https://medium.com/@amittidke/using-gitlab-cicd-variables-in-terraform-a21c011faa0a>

```

## -----
## Explication de la boucle, étape par étape
## -----
echo "$CLIENTS_JSON" | jq -c '.[[]]' | while read -r client; do
    username=$(echo "$client" | jq -r '.username')
    password=$(echo "$client" | jq -r '.password')
    MENU_OPTION=1 CLIENT="$username" PASS=2 PASSWORD="$password"
    ./openvpn-install.sh
done

## Point de départ: CLIENTS_JSON contient une chaîne JSON du type
## [{"username":"user1","password":"pass1"},
## {"username":"user2","password":"pass2"},...]
## C'est Terraform qui l'a injectée via jsonencode(openvpn_clients).

CLIENTS_JSON='[{"username":"user1","password":"pass1"},
{"username":"user2","password":"pass2"}]'

## -----
## Étape 1: echo "$CLIENTS_JSON"
## -----
## Envoie le JSON complet sur la sortie standard (stdout)
## pour qu'on puisse le passer à jq via un pipe |

## -----
## Étape 2: | jq -c '.[[]]'
## -----
## jq = parseur JSON en ligne de commande
## '.[[]]' = itère sur tous les éléments du tableau racine
## -c = "compact", sort chaque objet sur UNE SEULE ligne
##
## Résultat en sortie:
## {"username":"user1","password":"pass1"}
## {"username":"user2","password":"pass2"}
##
## Chaque objet devient une ligne indépendante, prête à être lue.

## -----
## Étape 3: while read -r client; do ... done
## -----
## read -r = lit UNE ligne depuis stdin et la stocke dans $client
## l'option -r évite que le backslash soit interprété
## while ... = continue tant qu'il reste des lignes à lire
##
## À chaque tour de boucle, $client contient un objet JSON:
## Tour 1: client='{"username":"user1","password":"pass1"}'
## Tour 2: client='{"username":"user2","password":"pass2"}'

## -----
## Étape 4: extraction des champs avec jq -r
## -----

```

```

username=$(echo "$client" | jq -r '.username')
password=$(echo "$client" | jq -r '.password')

## jq -r = "raw", sort la valeur SANS guillemets JSON autour
## Sans -r on aurait username='"user1"' au lieu de username='user1'
## $(...) = command substitution, capture la sortie dans la variable

## -----
## Étape 5: appel du script angristan avec les variables d'env
## -----
## MENU_OPTION=1 -> choix "add client" dans le menu du script
## CLIENT=...    -> nom du client à créer
## PASS=2        -> choix "mot de passe protégé" (au lieu de
passwordless)
## PASSWORD=...  -> mot de passe à utiliser
##
## Ces variables sont passées UNIQUEMENT à ce process (pas exportées
globalement)
## C'est la façon dont angristan permet de scripter son installeur sans
interaction.

## -----
## Résumé en une phrase
## -----
## On transforme un tableau JSON en lignes séparées, on lit chaque ligne
## une par une, on en extrait username/password, et on appelle le script
## OpenVPN avec ces valeurs pour créer chaque client automatiquement.

```

```

# Vérification
docker --version
docker compose version

# Terraform via Docker
sudo docker run --rm -it -v "$PWD:/workspace" -w /workspace
hashicorp/terraform:latest init
sudo docker run --rm -it -v "$PWD:/workspace" -w /workspace
hashicorp/terraform:latest plan -var-file="aws.tfvars"
sudo docker run --rm -it -v "$PWD:/workspace" -w /workspace
hashicorp/terraform:latest destroy -var-file="aws.tfvars"

# Docker Compose
sudo docker compose --file ./docker-compose.yml --project-name
hashicorp-terraform config
sudo docker compose --file ./docker-compose.yml --project-name
hashicorp-terraform up -d
sudo docker compose --file ./docker-compose.yml --project-name
hashicorp-terraform logs
sudo docker compose --file ./docker-compose.yml --project-name
hashicorp-terraform down

```

```
# Git clone via SSH en précisant la clé privée SSH
git clone -c "core.sshCommand=ssh -i ~/.ssh/aws-ci-cd-deploy-
gitlab.com" git@gitlab.com:AZIZI-Sajjad/cv.git

sync -e " ssh -i ~/.ssh/aws-ci-cd-deploy"
ubuntu@15.236.207.137:/home/ubuntu/cv.
zip .
```

#Commentaires

```
# SSH:
Clé publique dans le serveur distant ssh-copy-id
# Gitlab:
Clé publique dans github

# À faire :
Transmettre la clé privée de ssh de Gitlab dans le VPS EC2 de AWS
Avec Terraform ?
OU avec Gitla
```

```
# Créer le répertoire de destination
sudo mkdir -p "${deploy_directory}"

# Copier les fichiers compilés
sudo rsync -av --delete "${build_directory}/" "${deploy_directory}/"
```

PROF

```
sed -i 's/[[:space:]]\+$//' src/components/Skills.vue
sed -i '2c\ <q-timeline :layout="print || layout">'
src/components/Timeline.vue
sed -i 's/extendWebpack (cfg)/extendWebpack (/)' quasar.conf.js
sed -i -e '$a\' src/i18n/en-US/index.js
sed -i -e '$a\' src/i18n/fr-FR/index.js
```

```
while true; do
    ps aux | grep -Ei "git|apt|npm"
    printf '%0.s-' {1..133}; echo
    sudo ss -tulpn | grep -Ei "443|1191"
    printf '%0.s-' {1..133}; echo
    sudo /home/ubuntu/openvpn-install/openvpn-install.sh client list
    printf '%0.s-' {1..133}; echo
```


sleep 2

done